

Maestría y Doctorado en Educación matemática
Combinatoria y Probabilidad
Universidad Antonio Nariño - Sede Nicolás de Federman

Fredy Giovanni Quiazúa Merchán

Tres Problemas de Combinatoria
Profesor: EMERSON JULIAN LEON GUERRERO

17 de septiembre de 2025

Universidad Antonio Nariño

Solución de Ejercicios de Combinatoria y Probabilidad

Doctorado y Maestría de Educación Matemática



Ejercicio 1: Memorias USB



Ejercicio 2: Barras de Chocolate



Ejercicio 3: Corona de Oro

Presentación: Solución de Ejercicios

Introducción a los Problemas

Vamos a resolver tres ejercicios de combinatoria y probabilidad, aumentando gradualmente en dificultad.



Ejercicio 1: Memorias USB

Nivel: Fácil

Una empresa vende paquetes de memorias USB con capacidades en potencias de 2: 1GB, 2GB, 4GB, 8GB, 16GB y 32GB.

🎯 **Objetivo:** 40GB

⚠️ **Restricción:** Máximo 2 paquetes de cada tamaño



Ejercicio 2: Barras de Chocolate

Nivel: Intermedio

Una fábrica produce barras de chocolate con pesos en potencias de 2: 1g, 2g, 4g, 8g, 16g, 32g, 64g y 128g.

🎯 **Objetivo:** 260g

⚠️ **Restricción:** Máximo 3 barras de cada peso



Ejercicio 3: Corona de Oro

Nivel: Difícil

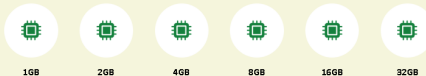
Un orfebre fabrica piezas de oro con pesos en potencias de 2: 1g, 2g, 4g, 8g, 16g, 32g, 64g, 128g y 256g.

🎯 **Objetivo:** 500g

⚠️ **Restricción:** Máximo 4 piezas de cada peso

Ejercicio 1: Memorias USB - Planteamiento

Una empresa vende paquetes de memorias USB con capacidades en potencias de 2. Un cliente necesita encargar exactamente 40GB, con la restricción de que puede pedir un máximo de dos paquetes de cada tamaño.



Objetivo

Encargar exactamente **40GB** de memorias USB



Restricción

Máximo **dos paquetes** de cada tamaño



Pregunta

¿De cuántas formas distintas puede realizar su pedido?

Ejercicio 1: Memorias USB - Solución

</> Enfoque Recursivo

Utilizamos un enfoque recursivo con backtracking para explorar todas las posibles combinaciones de memorias USB.

```
def buscar(indice, suma_actual, eleccion_actual):
    Si indice == len(tamamos):
        Si suma_actual == objetivo:
            soluciones.append(eleccion_actual.copy())
        return
    Para cantidad en range(3): # 0, 1, 2 paquetes
        nueva_suma = suma_actual + cantidad * tamamos[indice]
        Si nueva_suma <= objetivo:
            buscar(indice + 1, nueva_suma, eleccion_actual + [cantidad])
```

☰ Pasos del Algoritmo

- Inicialización:** Se establece como objetivo (40GB), tamanos (descendente) y soluciones (vacío).
- Caso Base:** Si indice llega a len(tamamos), se verifica si suma_actual es igual a objetivo. Si es así, se agrega eleccion_actual a soluciones.
- Paso Recursivo:** Para cada cantidad (0, 1, 2), se calcula nueva_suma y si no excede el objetivo, se hace una llamada recursiva con indice + 1.
- Orden descendente:** Al ordenar las capacidades de mayor a menor, el algoritmo puede terminar más rápido al encontrar soluciones válidas con menos recursiones.

☒ Soluciones Encontradas (11)

	32GB	16GB	8GB	4GB	2GB	1GB	Total
0	0	1	2	1	1	2	40
1	0	1	2	1	2	0	40
2	0	1	2	2	0	0	40
3	0	2	0	1	1	2	40
4	0	2	0	1	2	0	40
5	0	2	0	2	0	0	40

	32GB	16GB	8GB	4GB	2GB	1GB	Total
6	0	2	1	0	0	0	40
7	1	0	0	1	1	2	40
8	1	0	0	1	2	0	40
9	1	0	0	2	0	0	40
10	1	0	1	0	0	0	40

Ejercicio 2: Barras de Chocolate - Solución

</> Algoritmo Recursivo

Adaptación del enfoque recursivo con backtracking del ejercicio anterior:

- Exploramos todas las posibles combinaciones de barras de chocolate
- Comenzamos con la capacidad más grande (128g) y avanzamos hacia las más pequeñas (1g)
- Para cada peso, probamos con 0, 1, 2 o 3 barras (respecto a la restricción)
- Creamos una tabla con todas las combinaciones válidas que sumen exactamente 260g

📊 Ejemplos de Soluciones

Cantidad total de soluciones: 128

	128g	64g	32g	16g	8g	4g	2g	1g	Total
0	0	2	2	2	2	3	3	2	260
1	0	2	2	2	3	1	3	2	260
2	0	2	2	2	3	2	1	2	260
3	0	2	2	2	3	2	2	0	260
4	0	2	2	2	3	3	0	0	260

Cantidad total de soluciones: 128

...	...	...	...	...	...	...	...	...	...
123	1	2	0	0	0	2	0	0	260
124	1	2	0	0	0	1	0	0	260
125	2	0	0	0	0	0	1	2	260
126	2	0	0	0	0	0	2	0	260
127	2	0	0	0	0	1	0	0	260

[128 rows x 9 columns]

✅ Verificación

Verifiquemos una solución ejemplo:

$$2 \times 64g + 2 \times 32g + 2 \times 16g + 2 \times 8g + 3 \times 4g + 3 \times 2g + 2 \times 1g$$

$$= 128 + 64 + 32 + 16 + 12 + 6 + 2$$

$$= 260g$$



Resultados



El algoritmo encontró **128 combinaciones** válidas que suman exactamente 260g.

Ejercicio 3: Corona de Oro - Planteamiento



El Desafío Real

El rey necesita una corona que pese exactamente 500g de oro. El orfebre puede usar piezas de oro con pesos que son potencias de 2, con la restricción de que puede utilizar un máximo de cuatro piezas de cada peso.

Pesos Disponibles de Piezas de Oro

1g

(0ª
potencia de
2)

2g

(1ª
potencia de
2)

4g

(2ª
potencia de
2)

8g

(3ª
potencia de
2)

16g

(4ª
potencia de
2)

32g

(5ª
potencia de
2)

64g

(6ª
potencia de
2)

128g

(7ª
potencia de
2)

256g

(8ª
potencia de
2)



Objetivo

La corona debe pesar exactamente 500g de oro.



Restricción

El orfebre puede usar un máximo de cuatro piezas de cada peso.

★ Nivel: Difícil

Ejercicio 3: Corona de Oro - Solución

</> Enfoque de Programación Dinámica/Recursión

Cantidad total de soluciones: 1291

	256g	128g	64g	32g	16g	8g	4g	2g	1g	Total
0	0	0	4	4	4	3	4	4	4	500
1	0	0	4	4	4	4	2	4	4	500
2	0	0	4	4	4	4	3	2	4	500
3	0	0	4	4	4	4	3	3	2	500
4	0	0	4	4	4	4	3	4	0	500
...	...	...	...	...	...	...	...	...	...	...
1286	1	1	1	1	0	2	1	0	0	500
1287	1	1	1	1	1	0	0	0	4	500
1288	1	1	1	1	1	0	0	1	2	500
1289	1	1	1	1	1	0	0	2	0	500
1290	1	1	1	1	1	0	1	0	0	500

[1291 rows x 10 columns]



Resultados



El algoritmo encontró **1291** combinaciones válidas que suman exactamente 500 g.